

AWS Cloud Project Documentation

QuickLoan

Scalable Web Application Deployment on Amazon Web Services

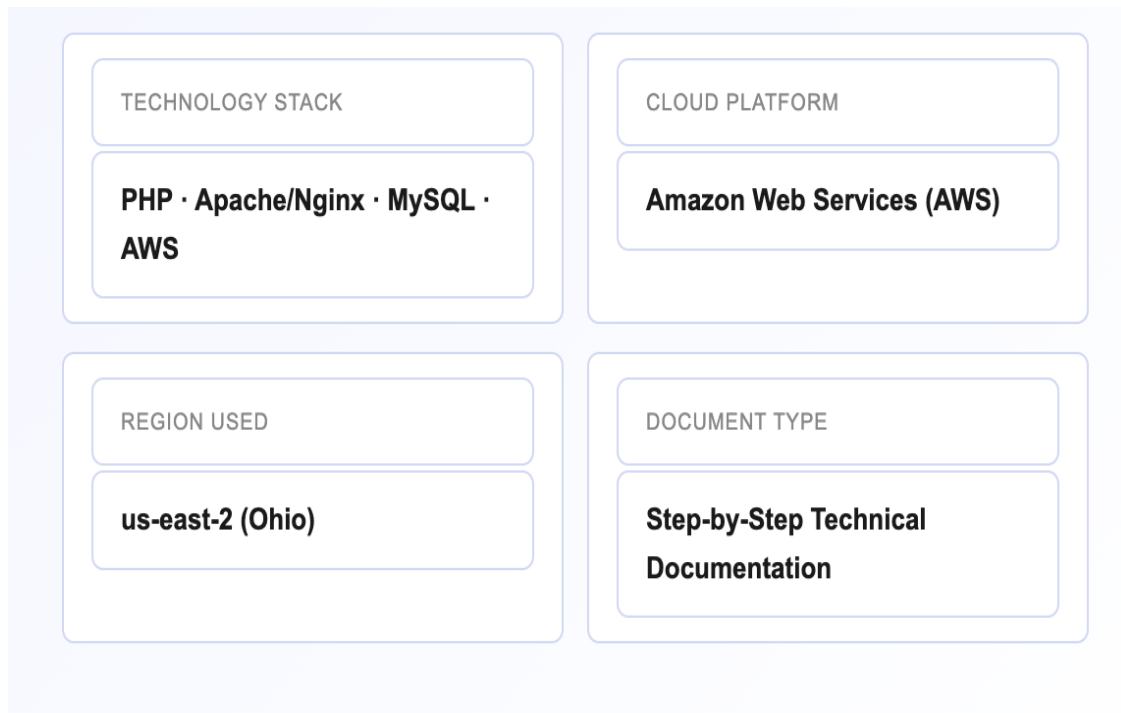
A complete cloud infrastructure project – VPC · EC2 · RDS · ALB · Auto Scaling · S3

Prepared by:

Miss. Sayali Dilip Chavan

Guided by:

Mr. Lekharaj (Trainer)



□ Table of Contents

1. Project Overview & Architecture
2. Technology Stack
3. Architecture Flow Diagram
4. Step 1 - VPC & Networking Setup
5. Step 2 - Security Groups Configuration
6. Step 3 - EC2 Instance Launch (Jump · App · Database Servers)
7. Step 4 - Application Deployment (Nginx + PHP)
8. Step 5 - S3 Bucket for Static Assets
9. Step 6 - RDS MySQL Database Setup
10. Step 7 - Domain Configuration (No-IP)
11. Step 8 - AMI Creation
12. Step 9 - Application Load Balancer & Target Group
13. Step 10 - Auto Scaling Group Configuration
14. Step 11 - End-to-End Verification
15. Roles & Responsibilities

Project Overview & Architecture

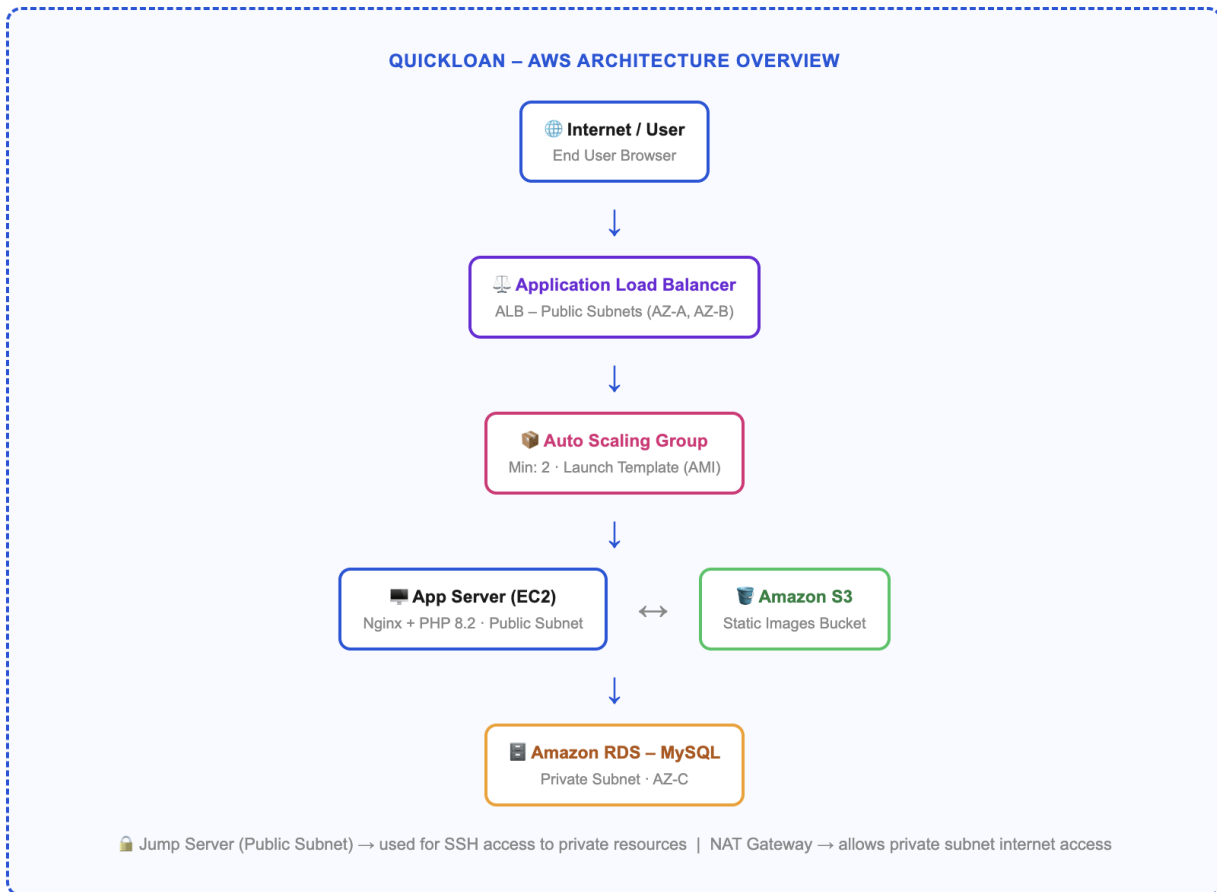
QuickLoan is a cloud-hosted loan application web platform built with a PHP backend and HTML/CSS/JavaScript frontend. The project demonstrates a production-grade AWS infrastructure setup encompassing networking, compute, storage, database, auto scaling, and load balancing — following industry best practices for high availability and fault tolerance.

The application was deployed inside a custom **Virtual Private Cloud (VPC)** with properly segmented public and private subnets across multiple Availability Zones. Traffic flows through an **Application Load Balancer (ALB)** to an **Auto Scaling Group** of EC2 instances, which communicate with a managed **RDS MySQL** database in a private subnet. Static assets (images) are served from **Amazon S3**.

Technology Stack

LAYER	TECHNOLOGY / SERVICE	PURPOSE
Frontend	HTML, CSS, JavaScript	User interface of the QuickLoan web app
Backend	PHP 8.2, php-fpm, php-mysqlnd	Server-side application logic
Web Server	Nginx	HTTP request handling and routing
Database	Amazon RDS – MySQL (MariaDB client)	Persistent storage of loan applications
Object Storage	Amazon S3	Hosting static image assets
Networking	VPC, Subnets, IGW, NAT Gateway, Route Tables	Isolated, secure cloud network
Compute	Amazon EC2 (Amazon Linux 2023)	Application, jump, and database servers
Scaling	Auto Scaling Group + Launch Template (AMI)	Horizontal scaling based on demand
Load Balancing	Application Load Balancer (ALB)	Traffic distribution across EC2 instances
DNS / Domain	No-IP (quickloan.hopto.org)	Free dynamic DNS for application access
File Transfer	WinSCP	Uploading application files to EC2

Architecture Flow Diagram



The flow begins with a user request hitting the **ALB**, which forwards it to one of the healthy EC2 instances managed by the **Auto Scaling Group**. Each EC2 instance runs **Nginx + PHP** and serves the QuickLoan frontend. Static images are loaded from **S3**. Application data (loan submissions) is stored in the **RDS MySQL** database in the private subnet. The **Jump Server** acts as the secure SSH entry point for all administrative access.

Step 1 – VPC & Networking Setup

1.1 Create VPC

A custom **Virtual Private Cloud (VPC)** was created to define an isolated network environment for the entire QuickLoan infrastructure. The VPC provides full control over IP address ranges, subnets, routing, and security – isolating our resources from other AWS accounts.

The screenshot displays the AWS Management Console interface for VPCs. At the top, there's a 'Your VPCs' section with a search bar and a table of VPCs. The table has columns for Name, VPC ID, State, Encryption controls, Encryption control mode, Block Public Access, and IPv4 CIDR. Two VPCs are listed: one with ID vpc-00968dfc408f4f905 and another with ID vpc-0f6e7174e6911dbd1, which is selected. Below the table, the details for the selected VPC are shown in a card format. The details include VPC ID, DNS resolution, Main network ACL, IPv6 CIDR, Encryption control ID, State, Tenancy, Default VPC, Network Address Usage metrics, Encryption control mode, Block Public Access, DHCP option set, IPv4 CIDR, Route 53 Resolver DNS Firewall rule groups, DNS hostnames, Main route table, IPv6 pool, and Owner ID.

Name	VPC ID	State	Encryption c...	Encryption control ...	Block Public...	IPv4 CIDR
-	vpc-00968dfc408f4f905	Available	-	-	Off	172.31.0.0/16
vpc1	vpc-0f6e7174e6911dbd1	Available	-	-	Off	10.10.0.0/16

vpc-0f6e7174e6911dbd1 / vpc1

Details

VPC ID vpc-0f6e7174e6911dbd1	State Available	Block Public Access Off	DNS hostnames Disabled
DNS resolution Enabled	Tenancy default	DHCP option set dopt-0d4e7bf59c1e5ed47	Main route table rtb-0bc6d716f3dfb3bd4
Main network ACL acl-0c2a83f4d99c39ba7	Default VPC No	IPv4 CIDR 10.10.0.0/16	IPv6 pool -
IPv6 CIDR -	Network Address Usage metrics Disabled	Route 53 Resolver DNS Firewall rule groups -	Owner ID 127358877080
Encryption control ID -	Encryption control mode -		

1.2 Create 3 Subnets (2 Public + 1 Private)

Three subnets were created within the VPC across multiple Availability Zones to support high availability:

- **Public Subnet 1** - Availability Zone A (hosts Jump Server & App Server, ALB node)
- **Public Subnet 2** - Availability Zone B (for ALB redundancy and Auto Scaling)
- **Private Subnet 3** - Availability Zone C (hosts RDS Database Server – no direct internet access)

Subnets (9) Info

Last updated 1 minute ago

Actions

Create subnet

Find subnets by attribute or tag

☐	Name	Subnet ID	State	VPC	Block Public...	IPv4 CIDR
☐	public-subnet1	subnet-09cced25d702bc147	Available	vpc-0f6e7174e6911dbd1 vpc1	Off	10.10.1.0/24
☐	RDS-Pvt-subnet-2	subnet-0439a7b8969999d08	Available	vpc-0f6e7174e6911dbd1 vpc1	Off	10.10.4.0/25
☐	private-subnet3	subnet-0d24e6814dafb1c85	Available	vpc-0f6e7174e6911dbd1 vpc1	Off	10.10.3.0/24
☐	-	subnet-0289fa81797fcd51f	Available	vpc-00968dfc408f4f905	Off	172.31.0.0/20
☐	public-subnet2	subnet-0965f6929e8f78841	Available	vpc-0f6e7174e6911dbd1 vpc1	Off	10.10.2.0/24

Select a subnet

1.3 Internet Gateway (IGW) – Attached to VPC

An **Internet Gateway** was created and attached to the VPC. The IGW enables resources in the public subnets (EC2 instances, ALB) to communicate directly with the internet, allowing users to access the QuickLoan web application.

Internet gateways (1/1) Info

Actions

Create internet gateway

Find internet gateways by attribute or tag

☒	Name	Internet gateway ID	State	VPC ID	Owner
☒	igw1	igw-0afb7dfe8b1533e78	Attached	vpc-0f6e7174e6911dbd1 vpc1	127358877080

igw-0afb7dfe8b1533e78 / igw1

Details

Tags

Details

Internet gateway ID	State	VPC ID	Owner
igw-0afb7dfe8b1533e78	Attached	vpc-0f6e7174e6911dbd1 vpc1	127358877080

1.4 NAT Gateway – Attached to Public Subnet 1

A **NAT Gateway** was deployed in Public Subnet 1 with an **Elastic IP** assigned to it. The NAT Gateway allows EC2 instances in the private subnet (e.g., the database server) to make outbound internet requests (e.g., for software updates) without being directly reachable from the internet – maintaining security.

NAT gateways (1/1) Info

Find NAT gateways by attribute or tag

Actions

Create NAT gateway

< 1 >

Name	NAT gateway ID	Connectivity...	State	State message	Availability ...	Route table ID	Primary public I
NAT-Gateway1	nat-013966bbc6abe3d8b	Public	Available	-	Zonal	-	13.58.6.157

nat-013966bbc6abe3d8b / NAT-Gateway1

Details

Secondary IPv4 addresses

Monitoring

Attachments

Tags

Details

NAT gateway ID

nat-013966bbc6abe3d8b

NAT gateway ARN

arn:aws:ec2:us-east-2:127358877080:natg
ateway/nat-013966bbc6abe3d8b

VPC

vpc-0f6e7174e6911dbd1 / vpc1

Connectivity type

Public

Primary public IPv4 address

13.58.6.157

Subnet

subnet-09cccd25d702bc147 / public-subnet1

State

Available

Primary private IPv4 address

10.10.1.47

Created

Sunday, March 15, 2026 at 14:44:11 GMT+
5:30

State message

-

Primary network interface ID

eni-01e245fdc84b176d9

Deleted

-

1.5 Route Tables – Public RT & Private RT

Two route tables were configured:

- **Public Route Table:** Associated with Public Subnet 1 and Public Subnet 2. A route was added pointing 0.0.0.0/0 to the Internet Gateway, enabling full internet access.
- **Private Route Table:** Associated with Private Subnet 3. A route was added pointing 0.0.0.0/0 to the NAT Gateway, enabling outbound-only internet access for the private subnet.

Route tables (6) Info

Last updated 2 minutes ago

Actions

Create route table

Find route tables by attribute or tag

< 1 >

Name	Route table ID	Explicit subnet associations	Edge associations	Main	VPC
RDS-Pvt-rt	rtb-050d2c1b16c57e009	3 subnets	-	No	vpc-0f6e7174e6911db
public-RT	rtb-0e195dec66155374	2 subnets	-	No	vpc-0f6e7174e6911db
private-RT	rtb-00a1dfed1977ff51d	subnet-0d24e6814dafb1c85 / private-subnet3	-	No	vpc-0f6e7174e6911db
-	rtb-039a2274b51451a24	-	-	Yes	vpc-00968dfc408f4f90

Select a route table

Two Security Groups were created to control inbound and outbound traffic to the EC2 instances and the RDS database, following the **principle of least privilege**.

SG-1 – App & Jump Server Security Group

This security group was attached to the Jump Server and App Server EC2 instances.

Inbound Rules:

- Port 22 (SSH) – for administrative remote access
- Port 80 (HTTP) – to allow web traffic to the application
- Port 3306 (MySQL/Aurora) – for Mysql

Security Groups (1/13) Info

Find security groups by attribute or tag

Actions Export security groups to CSV Create security group

	Name	Security group ID	Security group name	VPC ID	Description
☐	-	sg-0818d1b7be2855dcc	launch-wizard-3	vpc-00968dfc408f4f905	launch-wizard-3 created 2026-02-0
☐	-	sg-06c5eb67d46bee8c0	SG-15march-app	vpc-0f6e7174e6911dbd1	SG-15march-app
☐	-	sg-0a6c93f68f6b27848	default	vpc-0f6e7174e6911dbd1	default VPC security group
☐	-	sg-083f34727f69dbc82	ec2-rds-2	vpc-0f6e7174e6911dbd1	Security group attached to instance
☐	-	sg-07e9830cf3250816f	rds-ec2-1	vpc-00968dfc408f4f905	Security group attached to db1 to a
☐	-	sg-09ae203e9b2d5ba4e	ec2-rds-1	vpc-00968dfc408f4f905	Security group attached to instance
☐	-	sg-0181890d41d0cf494	default	vpc-00968dfc408f4f905	default VPC security group
☐	-	sg-05c876e3e72d5fd9	launch-wizard-1	vpc-00968dfc408f4f905	launch-wizard-1 created 2026-01-3
☐	-	sg-0f6e554cc46b7aafd	launch-wizard-2	vpc-00968dfc408f4f905	launch-wizard-2 created 2026-01-3
☒	-	sg-04d65efbdaa3782cd	SG-15march-database	vpc-0f6e7174e6911dbd1	SG-15march-database
☐	-	sg-079a1421470bf5340	launch-wizard-5	vpc-00968dfc408f4f905	launch-wizard-5 created 2026-02-0
☐	-	sg-0908eab188ba7ec30	rds-ec2-2	vpc-0f6e7174e6911dbd1	Security group attached to custome
☐	-	sg-0637f8c755fb276f6	launch-wizard-4	vpc-00968dfc408f4f905	launch-wizard-4 created 2026-02-0

sg-04d65efbdaa3782cd - SG-15march-database

Details

Security group name
SG-15march-database

Owner
127358877080

Security group ID
sg-04d65efbdaa3782cd

Inbound rules count
2 Permission entries

Description
SG-15march-database

Outbound rules count
1 Permission entry

VPC ID
vpc-0f6e7174e6911dbd1

SG-2 – Database Server Security Group

This security group was attached to the RDS MySQL instance to restrict access only to the application servers.

Inbound Rules:

- Port 22 (SSH) – for admin access via Jump Server
- Port 3306 (MySQL/Aurora) – source restricted to the App Server Security Group (SG-1), ensuring only application servers can connect to the database

This setup prevents direct database access from the public internet.

Step 3 – EC2 Instance Launch

Three EC2 instances were launched, each serving a distinct role in the infrastructure. All instances used **Amazon Linux 2023** as the base AMI.

Instance Name	Type	Subnet	Role
jump-server	Public	Public Subnet 1	Secure SSH gateway to reach private resources
app-server	Public	Public Subnet 1	Hosts Nginx + PHP 8.2 application
database-server	Private	Private Subnet 3	for RDS connection

Jump Server Setup

The Jump Server (also called a Bastion Host) was launched in the public subnet. Its only purpose is to act as a secure SSH relay — administrators SSH into the Jump Server first, then SSH from it to private EC2 instances or the database server using their private IP addresses. This eliminates the need to expose private servers to the internet.

SSH Access Pattern (Jumping Between Servers)

All server access was performed using the following SSH jump pattern: [For mac os]

```
# Step 1 – Login to Jump Server (Public IP)
ssh -i jumpserver.pem ec2-user@3.147.64.82

# Step 2 - From Jump Server, SSH into App Server (Private IP)
ssh -i app-server-keypair.pem ec2-user@10.10.2.211

# Step 2 (alt) - From Jump Server, SSH into Database Server (Private IP)
ssh -i database-server-keypair.pem ec2-user@10.10.3.229
```

Instances (3) Info

Find Instance by attribute or tag (case-sensitive)

All states

< 1 >

☐	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS
☐	app-server	i-036c04bfd6a9342f8	Running	t3.micro	3/3 checks passed	View alarms +	us-east-2b	-
☐	jump-server	i-018e4927ae71e3cb9	Running	t3.micro	3/3 checks passed	View alarms +	us-east-2a	-
☐	database-server	i-0967d41f6f73b43d5	Running	t3.micro	3/3 checks passed	View alarms +	us-east-2c	-

Select an instance

Find Instance by attribute or tag (case-sensitive)

All states

< 1 >

☐	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS
☐	jump-server	i-018e4927ae71e3cb9	Running	t3.micro	3/3 checks passed	View alarms +	us-east-2a	-
☐	database-server	i-0967d41f6f73b43d5	Running	t3.micro	3/3 checks passed	View alarms +	us-east-2c	-

i-036c04bfd6a9342f8 (app-server)

Instance ID

i-036c04bfd6a9342f8

IPv6 address

-

Hostname type

IP name: ip-10-10-2-211.us-east-2.compute.internal

Answer private resource DNS name

-

Auto-assigned IP address

18.219.237.110 [Public IP]

IAM role

-

IMDSv2

Required

Operator

Public IPv4 address

18.219.237.110 | open address

Instance state

Running

Private IP DNS name (IPv4 only)

ip-10-10-2-211.us-east-2.compute.internal

Instance type

t3.micro

VPC ID

vpc-0f6e7174e6911dbd1 (vpc1)

Subnet ID

subnet-0965f6929e8f78841 (public-subnet2)

Instance ARN

arn:aws:ec2:us-east-2:127358877080:instance/i-036c04bfd6a9342f8

Private IPv4 addresses

10.10.2.211

Public DNS

-

Elastic IP addresses

-

AWS Compute Optimizer finding

Opt-in to AWS Compute Optimizer for recommendations. | Learn more

Auto Scaling Group name

-

Managed

false

Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Instances (1/3) Info

Find Instance by attribute or tag (case-sensitive)

All states

< 1 >

☒	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS
☐	app-server	i-036c04bfd6a9342f8	Running	t3.micro	3/3 checks passed	View alarms +	us-east-2b	-
☒	jump-server	i-018e4927ae71e3cb9	Running	t3.micro	3/3 checks passed	View alarms +	us-east-2a	-
☐	database-server	i-0967d41f6f73b43d5	Running	t3.micro	3/3 checks passed	View alarms +	us-east-2c	-

i-018e4927ae71e3cb9 (jump-server)

Instance ID

i-018e4927ae71e3cb9

IPv6 address

-

Hostname type

IP name: ip-10-10-1-97.us-east-2.compute.internal

Answer private resource DNS name

-

Auto-assigned IP address

3.147.64.82 [Public IP]

IAM role

-

IMDSv2

Required

Operator

Public IPv4 address

3.147.64.82 | open address

Instance state

Running

Private IP DNS name (IPv4 only)

ip-10-10-1-97.us-east-2.compute.internal

Instance type

t3.micro

VPC ID

vpc-0f6e7174e6911dbd1 (vpc1)

Subnet ID

subnet-09cccd25d702bc147 (public-subnet1)

Instance ARN

arn:aws:ec2:us-east-2:127358877080:instance/i-018e4927ae71e3cb9

Private IPv4 addresses

10.10.1.97

Public DNS

-

Elastic IP addresses

-

AWS Compute Optimizer finding

Opt-in to AWS Compute Optimizer for recommendations. | Learn more

Auto Scaling Group name

-

Managed

false

Instances (1/3) [Info](#)

Find Instance by attribute or tag (case-sensitive) All states < 1 > ⚙️

☐	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS
☐	app-server	i-036c04bfd6a9342f8	Running	t3.micro	3/3 checks passed	View alarms +	us-east-2b	-
☐	jump-server	i-018e4927ae71e3cb9	Running	t3.micro	3/3 checks passed	View alarms +	us-east-2a	-
☒	database-server	i-0967d41f6f73b43d5	Running	t3.micro	3/3 checks passed	View alarms +	us-east-2c	-

i-0967d41f6f73b43d5 (database-server) ⚙️ ⌵

[Details](#) [Status and alarms](#) [Monitoring](#) [Security](#) [Networking](#) [Storage](#) [Tags](#)

▼ **Instance summary** [Info](#)

Instance ID i-0967d41f6f73b43d5	Public IPv4 address -	Private IPv4 addresses 10.10.3.229
IPv6 address -	Instance state Running	Public DNS -
Hostname type IP name: ip-10-10-3-229.us-east-2.compute.internal	Private IP DNS name (IPv4 only) ip-10-10-3-229.us-east-2.compute.internal	Elastic IP addresses -
Answer private resource DNS name -	Instance type t3.micro	AWS Compute Optimizer finding Opt-in to AWS Compute Optimizer for recommendations. Learn more
Auto-assigned IP address -	VPC ID vpc-0f6e7174e6911dbd1 (vpc1)	Auto Scaling Group name -
IAM role -	Subnet ID subnet-0d24e6814dafb1c85 (private-subnet3)	

© 2026, Amazon Web Services, Inc. or its affiliates. [Privacy](#) [Terms](#) [Cookie preferences](#)

Step 4 – Application Deployment on App Server

The QuickLoan PHP application was deployed on the App Server by installing Nginx as the web server and PHP 8.2 as the runtime engine. All commands were executed after SSH-ing into the app-server via the Jump Server.

4.1 System Update & Nginx Installation

The server packages were first updated, and Nginx was installed and configured to start automatically on reboot.

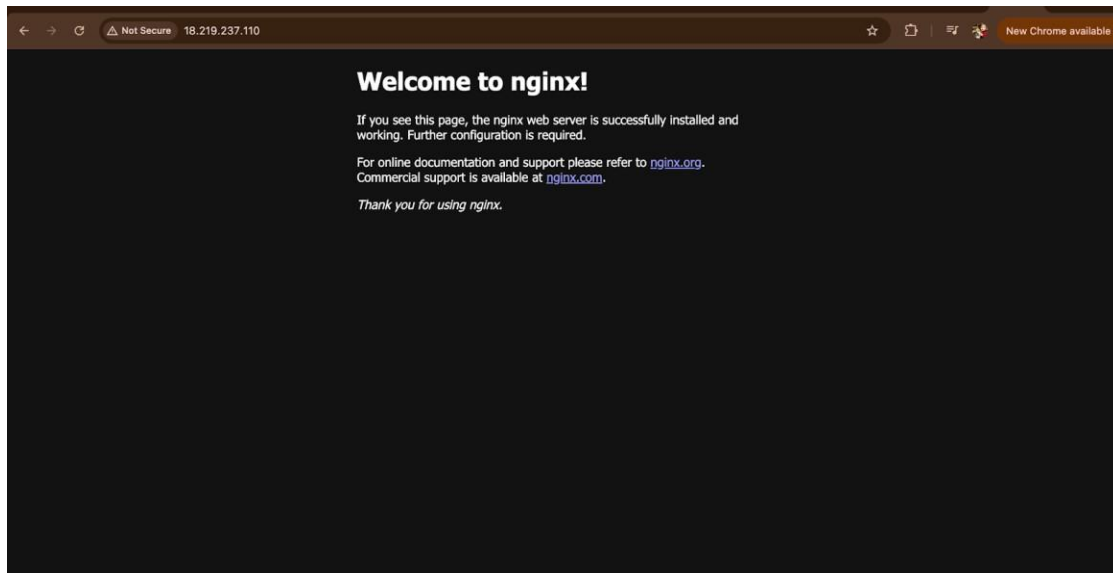
```
yum update -y

yum install -y nginx

systemctl start nginx

chkconfig nginx on

# Verify: Open browser → http://<public-ip>:80 – should show Nginx default page
```



4.2 PHP 8.2 Installation

PHP 8.2 along with required extensions (php-fpm for process management, php-mysqlnd for MySQL connectivity) was installed and started.

```
yum install -y php8.2 php-fpm php-mysqlnd php-pdo php-mbstring
php -v
systemctl start php-fpm
chkconfig php-fpm on
systemctl restart nginx
```

4.3 Upload Application Files via WinSCP

The application source files were transferred from the local laptop to the app server using **WinSCP** (a GUI-based SCP/SFTP tool). Three folders were uploaded to the EC2 home directory (`/home/ec2-user/`):

- `includes/` – PHP backend files (DB connection, config)
- `nginx/` – Custom Nginx virtual host configuration
- `public/` – Frontend HTML, CSS, JS, and PHP files

4.4 Move Files to Nginx Web Root & Set Permissions

The uploaded folders were moved to the Nginx web root directory, and proper ownership and permissions were set so Nginx can read and serve the files.

```
mv /home/ec2-user/includes /usr/share/nginx/html

mv /home/ec2-user/public /usr/share/nginx/html

# Set Nginx as owner of the public folder

chown -R nginx:nginx /usr/share/nginx/html/public

chmod -R 755 /usr/share/nginx/html/public
```

```
-rw----- 1 ec2-user ec2-user 610 Mar 15 12:22 .bash_history
[ec2-user@ip-10-10-2-211 ~]$ ls -lart /usr/share/nginx/html
total 16
-rw-r--r-- 1 root root 368 Feb 4 21:54 nginx-logo.png
-rw-r--r-- 1 root root 3693 Feb 4 21:54 50x.html
-rw-r--r-- 1 root root 3650 Feb 4 21:54 404.html
lrwxrwxrwx 1 root root 14 Feb 4 21:55 poweredby.png -> nginx-logo.png
-rw-r--r-- 1 root root 615 Feb 4 21:55 index.html
drwxr-xr-x 4 root root 33 Mar 15 10:01 ..
drwxr-xr-x 2 root root 27 Mar 15 10:01 icons
drwxr-xr-x 5 root root 142 Mar 15 10:31 .
drwxr-xr-x 3 nginx nginx 103 Mar 15 11:01 public
drwxr-xr-x 2 ec2-user ec2-user 28 Mar 15 11:07 includes
[ec2-user@ip-10-10-2-211 ~]$
```

Step 5 – Amazon S3 Bucket for Static Assets

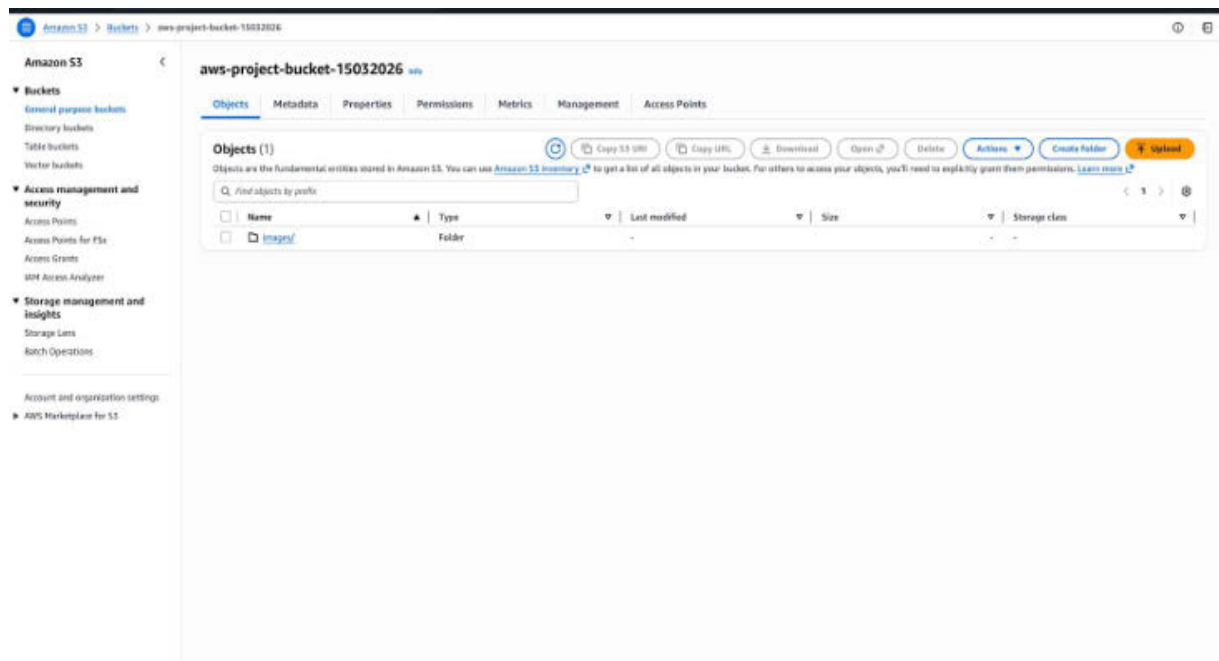
5.1 Create S3 Bucket

An S3 bucket was created in the **us-east-2 (Ohio)** region to store and serve static image assets used by the QuickLoan frontend. Hosting images on S3 offloads static content delivery from the application servers, improving performance and reducing compute costs.

Bucket Name: aws-project-bucket-15032026

5.2 Upload Images Folder to S3

The `images/` folder (containing the QuickLoan logo and other assets) was uploaded to the S3 bucket. The bucket was configured to allow public read access for serving images directly to the browser.



5.3 Update Image Paths in Application Files

The application source files were updated to reference the new S3 bucket URL instead of the old bucket path.

```
# Update index.html — Replace old S3 URL with new bucket URL

vim /usr/share/nginx/html/public/index.html

# Inside vim, run the substitution:

:%s/loanapp332211.s3.eu-west-2.amazonaws.com/aws-project-bucket-27022026.s3.us-east-2.amazonaws.com

# Update apply.php — Manually update the image src tag

vim /usr/share/nginx/html/public/apply.php

# Replace image src with:


```

```

</head>
<body>
  <header>
    <div class="logo">
      
    </div>
    <nav>
      <a href="apply.php" class="apply-btn">Apply for Loan</a>
    </nav>
  </header>
  <section class="hero">
    <h1>Your Financial Dreams, Realized.</h1>
    <p>QuickLoan provides fast, reliable, and affordable financial solutions.</p>
  </section>
  <section class="products">
    <h2>Our Loan Products</h2>
    <div class="product-grid">
      <div class="product">
        
        <h3>Home Loans</h3>
        <p>Own your dream home with our flexible home loan options.</p>
      </div>
      <div class="product">
        
        <h3>Gold Loans</h3>
        <p>Get instant cash with our hassle-free gold loan services.</p>
      </div>
      <div class="product">
        
        <h3>Vehicle Loans</h3>
        <p>Drive your dream vehicle with our easy vehicle loan plans.</p>
      </div>
      <div class="product">
        
        <h3>Personal Loans</h3>
        <p>Meet your urgent financial needs with our quick personal loans.</p>
      </div>
    </div>
  </section>
</body>
</html>

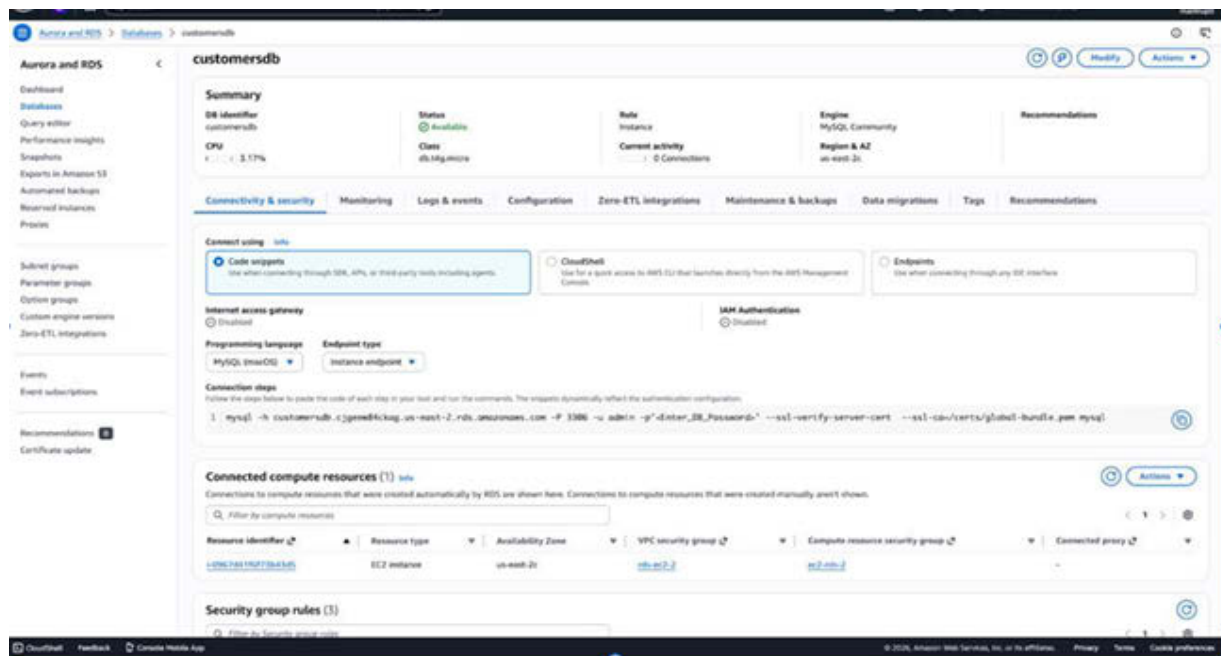
```

"/usr/share/nginx/html/public/index.html" [readonly][dos] 56L, 3087B 41,17 33%

Step 6 – Amazon RDS MySQL Database Setup

6.1 Create RDS MySQL Instance

An **Amazon RDS MySQL** instance was provisioned in the **Private Subnet (AZ-C)**. RDS was chosen over a self-managed MySQL server for its managed backups, automated patching, and high availability features. The database security group (SG-2) was attached to restrict access only to the app server.



6.2 Update DB Connection Configuration on App Server

The database connection file on the app server was updated with the RDS endpoint, username, and password.

```
cd /usr/share/nginx/html/includes
vim db_connect.php

# Replace the following values:

hostname = customersdb.cjgeew84ckag.us-east-2.rds.amazonaws.com

username = admin

password = admin123
```

6.3 Create Database & Table via Jump → Database Server

Using the SSH jump pattern, the database server was accessed and the MySQL client was used to connect to RDS and create the required schema.

```
# SSH from Jump Server to Database Server

ssh -i database-server-keypair.pem ec2-user@10.10.3.229

# Install MySQL client

yum install mariadb105 -y
```

```

# Connect to RDS

mysql -h customersdb.cjgeew84ckag.us-east-2.rds.amazonaws.com -u admin -padmin123

# Create database and table

CREATE DATABASE quickloan_db;

USE quickloan_db;

CREATE TABLE applications (

    id INT AUTO_INCREMENT PRIMARY KEY,

    name VARCHAR(255) NOT NULL,

    contact VARCHAR(20) NOT NULL,

    email VARCHAR(255) NOT NULL,

    loan_type VARCHAR(50) NOT NULL

);

```

```

mysql [(none)]> show databases;
ERROR 1046 (3D000): No database selected
MySQL [(none)]> show databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| quickloan_db |
| sys |
+-----+
5 rows in set (0.014 sec)

MySQL [(none)]> use quickloan_db;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
MySQL [quickloan_db]> show tables;
+-----+
| Tables_in_quickloan_db |
+-----+
| applications |
+-----+
1 row in set (0.001 sec)

MySQL [quickloan_db]>

```

```
MySQL [quickloan_db]> select * from applications;
+-----+-----+-----+-----+-----+
| id | name   | contact | email           | loan_type |
+-----+-----+-----+-----+-----+
| 1 | Trupti | 0000000000 | trupti@gmail.com | Gold Loan |
| 2 | Riya   | 1111111111 | riya@gmail.com   | Home Loan |
| 3 | Shraddha | 2222222222 | Shraddha@gmail.com | Vehicle Loan |
| 4 | Supriya | 5555555555 | supriya555@gmail.com | Home Loan |
+-----+-----+-----+-----+-----+
4 rows in set (0.001 sec)

MySQL [quickloan_db]> 
```

Step 7 – Domain Configuration via No-IP

7.1 Register Free Dynamic DNS Domain

A free dynamic DNS hostname was registered at **noip.com** to give the application a human-readable URL instead of an IP address. The domain `domainname.freedynamicdns.net` was configured to point to the app server's public IP.

7.2 Update Nginx Virtual Host Configuration

The Nginx configuration file was updated with the new domain name and moved to the Nginx configuration directory.

```
# SSH into app server, edit the virtual host config

vim /home/ec2-user/nginx/quickloan.conf

# Set server_name directive:

server_name domainname.freedynamicdns.net;

# Move config to Nginx conf directory

mv /home/ec2-user/nginx/quickloan.conf /etc/nginx/conf.d

# Test Nginx config and restart

nginx -t

systemctl restart nginx
```

```
systemctl reload nginx
```

Records

My No-IP > Records

Create Hostname

Free Dynamic DNS: 1/1

Filter by Zone

Upgrade to Enhanced Dynamic DNS

Free Dynamic DNS accounts are limited to 1 Hostname, currently there is 1 Hostname in your account.

Upgrade Now

freedynamicdns.net 1 record

No-IP Owned

Add Record

Name	Type	Content	TTL	Last Update
domainname	CNAME	LB1-468236150.us-east-2.elb.amazonaws.com	60	Mar 15, 2026 07:24:55

my@c256b77d3 2026-03-12T19:02:57Z web03

Records

My No-IP > Records

Create Hostname

Free Dynamic DNS: 1/1

Filter by Zone

Upgrade to Enhanced Dynamic DNS

Free Dynamic DNS accounts are limited to 1 Hostname, currently there is 1 Hostname in your account.

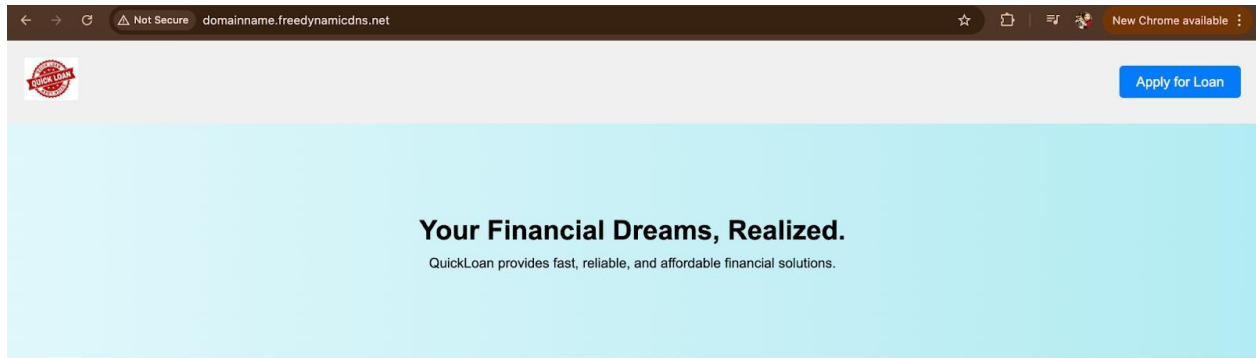
Upgrade Now

freedynamicdns.net 1 record

No-IP Owned

Add Record

Name	Type	Content	TTL	Last Update
domainname	A	18.219.237.110	60	Mar 15, 2026 05:08:21



Our Loan Products



Home Loans

Own your dream home with our flexible home loan options.



Gold Loans

Get instant cash with our hassle-free gold loan services.



Vehicle Loans

Drive your dream vehicle with our easy vehicle loan plans.



Personal Loans

Meet your urgent financial needs with our quick personal loans.

Step 8 – AMI (Amazon Machine Image) Creation

8.1 Create AMI from the Configured App Server

Once the application was fully deployed and tested on the app server, a **custom AMI (Amazon Machine Image)** was created from that EC2 instance. This AMI is essentially a complete snapshot of the configured server — including the OS, Nginx, PHP, application code, and all configurations.

The AMI serves as the **golden image** for the Auto Scaling Group — every new instance launched by Auto Scaling will be an exact replica of the tested and working app server, ensuring consistency across all instances.

Amazon Machine Images (AMIs) (1/1) Info

Recycle Bin

EC2 Image Builder

Actions

Launch instance from AMI

Owned by me

Find AMI by attribute or tag

1

Name	AMI name	AMI ID	Source	Owner	Visibility	Status
app-server-image	ami-03910959269fc5904	127358877080/app-server-image	127358877080	Private	Available	

AMI ID: ami-03910959269fc5904

Details

Permissions

Storage

My AMI usage

AMI ancestry

Tags

AMI ID ami-03910959269fc5904	Image type machine	Platform details Linux/UNIX	Root device type EBS
AMI name app-server-image	Owner account ID 127358877080	Architecture x86_64	Usage operation RunInstances
Root device name /dev/xvda	Status Available	Source 127358877080/app-server-image	Virtualization type hvm
Boot mode uefi-preferred	State reason -	Creation date 2026-03-15T12:38:14.000Z	Kernel ID -
Description app-server-image	Product codes -	RAM disk ID -	Deprecation time -
Last launched time -	Block devices /dev/xvda=snap-0265bc60a09646f28:8:true:gp3	Deregistration protection Disabled	Allowed image -
Source AMI ID ami-0b0b78dcacbab728f	Source AMI Region us-east-2		

Step 9 – Application Load Balancer & Target Group

9.1 Create Target Group

A **Target Group** was created to register the EC2 instances that the Load Balancer will route traffic to. The target group was configured with:

- **Target Type:** Instance
- **Protocol:** HTTP, Port 80
- **Health Checks:** HTTP on root path (/) to verify instance availability

9.2 Create Application Load Balancer (ALB)

An **Application Load Balancer** was provisioned in front of the Auto Scaling Group. The ALB was configured as:

- **Scheme:** Internet-facing (publicly accessible)
- **Subnets:** Both Public Subnet 1 (AZ-A) and Public Subnet 2 (AZ-B) – for multi-AZ availability
- **Listener:** HTTP on Port 80 → forwards to the Target Group

- **Security Group:** App Server SG (allowing HTTP/HTTPS traffic)

Load balancers (1/1) [What's new?](#)

Create load balancer

Elastic Load Balancing scales your load balancer capacity automatically in response to changes in incoming traffic.

☒	Name	State	Type	Scheme	IP address type	VPC ID	Availability Zones	Security Groups
☒	LB1	Active	application	Internet-facing	IPv4	vpc-0f6e7174e6911dbd1	2 Availability Zones	sg-06c...

Load balancer: LB1

[Details](#)

[Listeners and rules](#)

[Network mapping](#)

[Resource map](#)

[Security](#)

[Monitoring](#)

[Integrations](#)

[Attributes](#)

[Capacity](#)

[Tags](#)

Details

Load balancer type
Application

Status
✔ Active

Scheme
Internet-facing

Hosted zone
Z3AADJGX6KTTL2

VPC
[vpc-0f6e7174e6911dbd1](#)

Availability Zones
[subnet-09cccd25d702bc147](#) us-east-2a (use2-az1)
[subnet-0965f6929e8f78841](#) us-east-2b (use2-az2)

Load balancer IP address type
IPv4

Date created
March 15, 2026, 18:16 (UTC+05:30)

Load balancer ARN
arn:aws:elasticloadbalancing:us-east-2:127358877080:loadbalancer/app/LB1/a8de0a7d8e4830bd

DNS name [Info](#)
LB1-468236150.us-east-2.elb.amazonaws.com (A Record)

9.3 Stop Manually Launched Instances

After the Load Balancer and Auto Scaling Group were configured, the manually launched **app-server** and **jump-server** EC2 instances were stopped. Going forward, all traffic is handled by the Auto Scaling Group instances behind the ALB — the jump server remains as an administrative access point only when needed.

Step 10 – Auto Scaling Group Configuration

10.1 Create Launch Template from AMI

A **Launch Template** was created using the custom AMI created in Step 8. The launch template defines the configuration for each new EC2 instance that Auto Scaling will launch, including:

- AMI ID (our custom QuickLoan AMI)
- Instance Type (e.g., t2.micro / t3.micro)
- Security Group (App Server SG)
- Key Pair for SSH access

- ## Launch Templates (1/1) [Info](#)

Actions ▾
Create launch template

☒	Launch Template ID ▾	Launch Template Name ▾	Default Version ▾	Latest Version ▾	Create Time ▾	Created By ▾
☒	lt-08edac3b6ff036088	LT-15march	1	1	2026-03-15T13:01:58.000Z	arn:aws:iam::127358877080:root

LT-15march (lt-08edac3b6ff036088)

▾

Launch template details

Actions ▾
Delete template

Launch template ID
 lt-08edac3b6ff036088

Launch template name
 LT-15march

Default version
 1

Owner
 arn:aws:iam::127358877080:root

Details
Versions
Template tags

Launch template version details

Actions ▾
Delete template version

Version

1 (Default) ▾

Description
 LT-15march

Date created
 2026-03-15T13:01:58.000Z

Created by
 arn:aws:iam::127358877080:root

Instance details
Storage
Resource tags
Network interfaces
Advanced details

AMI ID
 ami-03910959269fc5904

Instance type
 t2.micro

Availability Zone
 -

Key pair name
 LT-keypair

Security groups
 -

Security group IDs
 sg-06c5eb67d46bee8c0

Availability Zone Id
 -

The screenshot displays the AWS Management Console interface for an Auto Scaling group named 'ASG-15march'. The top section shows the group's details, including its name, launch template ('LT-15march'), version ('Version Default'), and current state (2 instances, 2 desired capacity, 2 min, 5 max, 2 available). Below this, the 'Auto Scaling group: ASG-15march' section shows the 'Send to' dropdown set to 'On instance action' and a 'Create notification' button. The 'Activity history (2)' section shows two successful activities, both indicating a change in desired capacity from 0 to 2 and the start of a new instance.

Name	Launch template/configuration	Instances	Status	Desired capacity	Min	Max	Availabi
ASG-15march	LT-15march Version Default	2	-	2	2	5	2 Availa

Auto Scaling group: ASG-15march

Send to: On instance action

No notifications are currently specified

Create notification

Activity history (2)

Status	Description	Cause	Start time	End time
Successful	Launching a new EC2 instance: i-0c1fcea0291f360ad	At 2026-03-15T13:07:34Z a user request created an AutoScalingGroup changing the desired capacity from 0 to 2. At 2026-03-15T13:08:42Z an instance was started in response to a difference between desired and actual capacity, increasing the capacity from 0 to 2.	2026 March 15, 06:38:44 PM +05:30	2026 March 15, 06:39:00 PM +05:30
Successful	Launching a new EC2 instance: i-08afb39fbb7f8306	At 2026-03-15T13:07:34Z a user request created an AutoScalingGroup changing the desired capacity from 0 to 2. At 2026-03-15T13:08:42Z an instance was started in response to a difference between desired and actual capacity, increasing the capacity from 0 to 2.	2026 March 15, 06:38:44 PM +05:30	2026 March 15, 06:39:01 PM +05:30

Step 11 – End-to-End Verification

11.1 Application Access Test

The QuickLoan web application was accessed via the ALB DNS name and the No-IP domain in a browser. The homepage loaded correctly with the QuickLoan logo being served from S3 and the application pages rendering as expected.

11.2 Loan Application Form Submission Test

A test loan application was submitted through the web form to verify the complete data flow — from the frontend form to the PHP backend, and finally stored in the RDS MySQL database.



Apply for Loan

Name:

Contact Number:

Email Address:

Type of Loan:

11.3 Database Entry Verification

After submitting the form, the data entry was verified directly in the RDS database using the MySQL client:

```
mysql -h customersdb.cjgeew84ckag.us-east-2.rds.amazonaws.com -u admin -padmin123

USE quickloan_db;

SELECT * FROM applications;

# Output showed the latest submitted entry – confirming end-to-end data flow
```

```
MySQL [quickloan_db]> show tables;
+-----+
| Tables_in_quickloan_db |
+-----+
| applications            |
+-----+
1 row in set (0.001 sec)

MySQL [quickloan_db]> select * from applications;
+-----+-----+-----+-----+-----+
| id | name   | contact | email          | loan_type |
+-----+-----+-----+-----+-----+
| 1 | Trupti | 0000000000 | trupti@gmail.com | Gold Loan |
| 2 | Riya   | 1111111111 | riya@gmail.com   | Home Loan |
| 3 | Shraddha | 2222222222 | Shraddha@gmail.com | Vehicle Loan |
| 4 | Supriya | 5555555555 | supriya555@gmail.com | Home Loan |
| 5 | Renuka | 1234567891 | renuka@gmail.com | Personal Loan |
+-----+-----+-----+-----+-----+
5 rows in set (0.003 sec)

MySQL [quickloan_db]> █
```

Step 12 – CloudWatch Alarms & SNS Email Notifications

To proactively monitor the health of the QuickLoan infrastructure, **Amazon CloudWatch Alarms** were configured with **Amazon SNS (Simple Notification Service)** to send automated email alerts when key metrics crossed defined thresholds.

12.1 Create SNS Topic for Email Notifications

Before creating alarms, an **SNS Topic** was created to act as the notification channel.

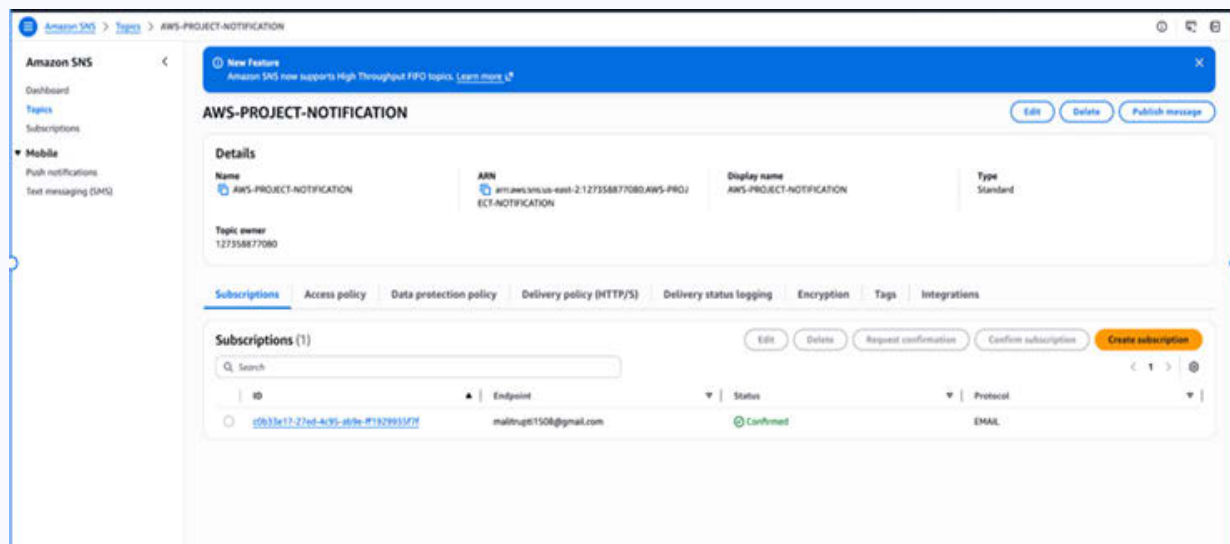
Navigate to **SNS** → **Topics** → **Create Topic**:

- **Type:** Standard
- **Topic Name:** e.g., quickloan-alerts

After creating the topic, a **Subscription** was added:

- **Protocol:** Email
- **Endpoint:** your email address (e.g., yourname@gmail.com)

A confirmation email was sent to the address — the subscription link in that email was clicked to **confirm the subscription**. Once confirmed, this SNS topic is ready to deliver alarm notifications directly to the inbox.

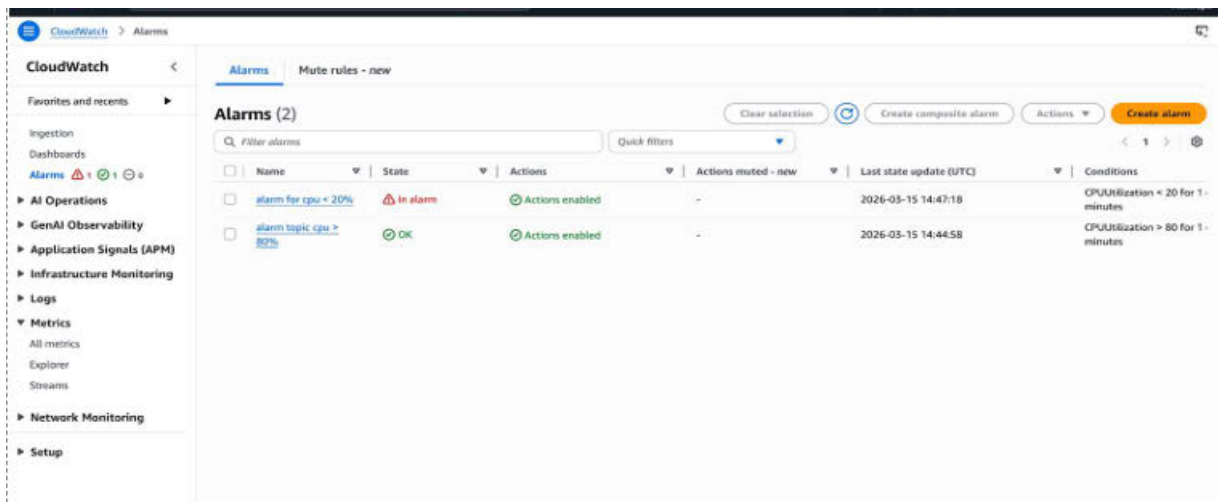


12.2 Create CloudWatch Alarm – High CPU Utilization

Navigate to **CloudWatch** → **Alarms** → **Create Alarm**:

- **Metric:** EC2 → Per-Instance Metrics → CPUUtilization
- **Statistic:** Average | **Period:** 5 minutes
- **Condition:** Greater than 70% (threshold)
- **Alarm State:** ALARM
- **Notification:** Send notification to SNS topic quickloan-alerts
- **Alarm Name:** e.g., QuickLoan-HighCPU-Alarm

When CPU utilization of any EC2 instance exceeds 70%, an alarm is triggered and an **email notification is automatically sent** to the configured address – alerting the team to investigate or manually scale.



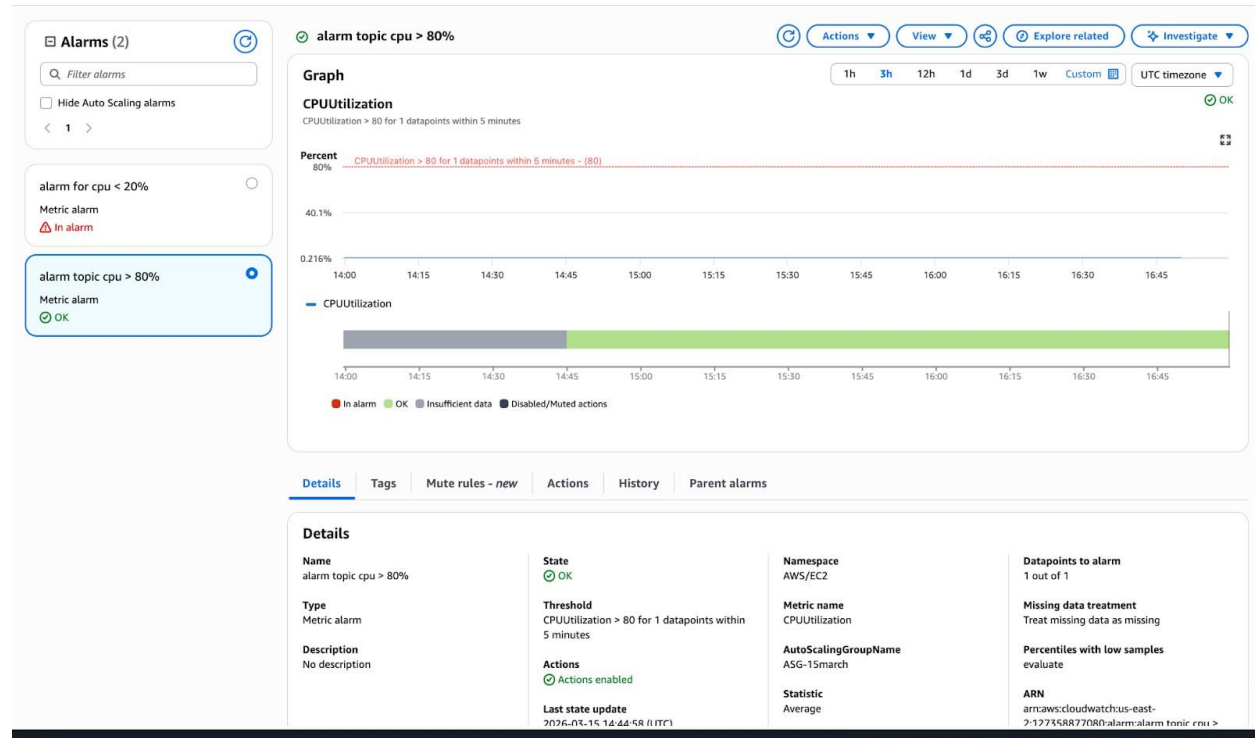
12.3 Create CloudWatch Alarm – Auto Scaling Trigger (Scale Out)

A second alarm was configured to trigger **Auto Scaling scale-out** (add instances) when load increases:

- **Metric:** Auto Scaling Group → CPUUtilization (average across group)
- **Condition:** Greater than 80% for 2 consecutive periods

- **Action:** Execute Auto Scaling Policy → *Add 1 Instance*
- **Also notify:** SNS topic `quickloan-alerts` (email sent on scale-out event)

This ensures the application automatically handles traffic spikes by adding EC2 instances while also notifying the team of scaling activity.

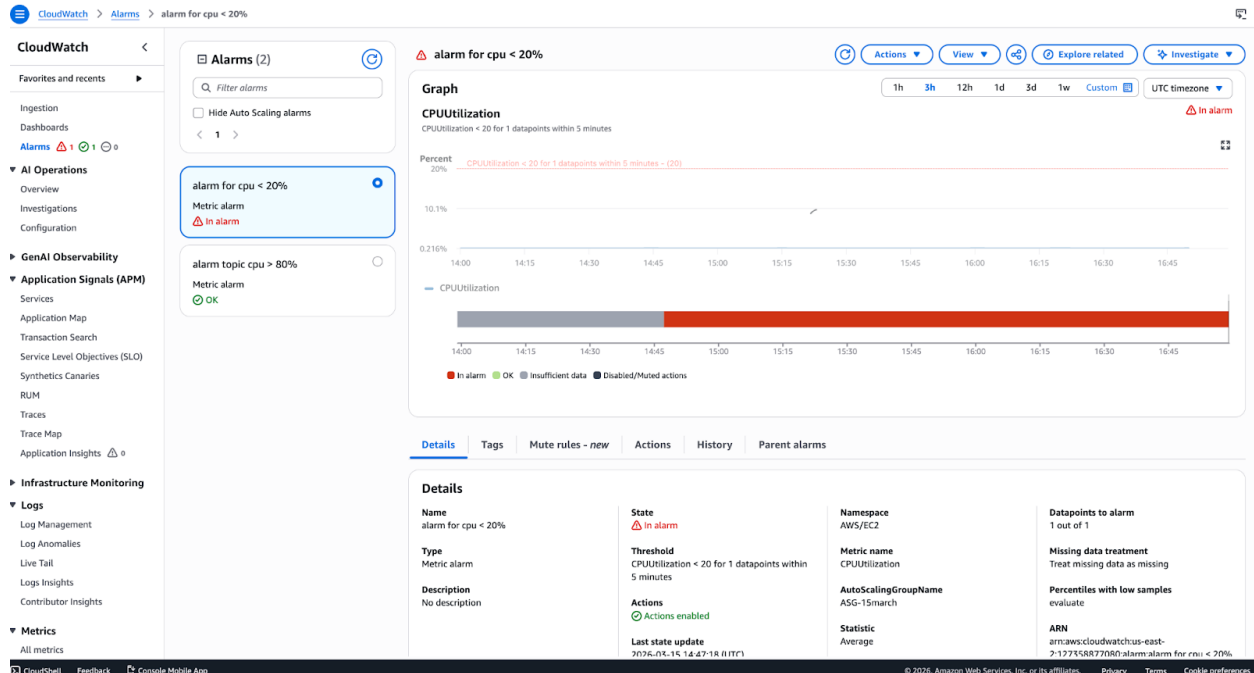


12.4 Create CloudWatch Alarm – Auto Scaling Trigger (Scale In)

A scale-in alarm was also configured to remove instances during low-traffic periods to reduce cost:

- **Metric:** Auto Scaling Group → CPUUtilization
- **Condition:** Less than 20% for 2 consecutive periods
- **Action:** Execute Auto Scaling Policy → *Remove 1 Instance*
- **Notification:** SNS topic `quickloan-alerts` (email sent on scale-in event)

This completes a full **dynamic scaling loop** – the infrastructure grows under load and contracts during idle periods, optimizing both performance and cost.



12.5 Email Notification Verification

After triggering a test condition (e.g., running a CPU stress test on an EC2 instance), the CloudWatch Alarm entered the **ALARM** state and an **automated email notification** was received at the configured email address via SNS. The email contained:

- Alarm name and description
- Current metric value that triggered the alarm
- Threshold that was breached
- Timestamp of the alarm
- Link to view the alarm in the AWS Console

ALARM: "alarm for cpu < 20%" in US East (Ohio)

Inbox x

AWS-PROJECT-NOTIFICATION

<no-reply@sns.amazonaws.com>

8:17 PM (4 minutes ago)

☆

😊

↶

⋮

to me ▾

You are receiving this email because your Amazon CloudWatch Alarm "alarm for cpu < 20%" in the US East (Ohio) region has entered the ALARM state, because "Threshold Crossed: 1 out of the last 1 datapoints [0.21695841437436267 (15/03/26 14:37:00)] was less than the threshold (20.0) (minimum 1 datapoint for OK -> ALARM transition)." at "Sunday 15 March, 2026 14:47:18 UTC".

View this alarm in the AWS Management Console:
<https://us-east-2.console.aws.amazon.com/cloudwatch/deeplink.js?region=us-east-2#alarmsV2:alarm/arn:aws:cloudwatch:us-east-2:127358877080:alarm:alarm%20for%20cpu%20%3C%2020%25>

Alarm Details:

- Name: alarm for cpu < 20%
- Description:
- State Change: INSUFFICIENT_DATA -> ALARM
- Reason for State Change: Threshold Crossed: 1 out of the last 1 datapoints [0.21695841437436267 (15/03/26 14:37:00)] was less than the threshold (20.0) (minimum 1 datapoint for OK -> ALARM transition).
- Timestamp: Sunday 15 March, 2026 14:47:18 UTC
- AWS Account: 127358877080
- Alarm Arn: arn:aws:cloudwatch:us-east-2:127358877080:alarm:alarm for cpu < 20%

Threshold:

- The alarm is in the ALARM state when the metric is LessThanThreshold 20.0 for at least 1 of the last 1 period(s) of 300 seconds.

Monitored Metric:

- MetricNamespace: AWS/EC2
- MetricName: CPUUtilization
- Dimensions: [AutoScalingGroupName = ASG-15march]
- Period: 300 seconds
- Statistic: Average
- Unit: not specified

↶ Reply

↷ Forward

😊

Roles & Responsibilities

#	Responsibility
1	Designed and created the VPC, subnets (public/private), route tables, Internet Gateway, and NAT Gateway.
2	Configured Security Groups to allow only necessary traffic – port 80/443 for EC2 instances and port 3306 for RDS, source-restricted to the App Server SG.
3	Deployed and tested the PHP application on EC2 with Nginx and PHP-FPM; created a custom AMI after successful end-to-end testing.
4	Configured the S3 bucket for static asset hosting and updated application source files to reference the correct S3 URLs.

- 5 Provisioned and configured Amazon RDS MySQL; created the application database schema and verified connectivity from the app server.
- 6 Configured Launch Templates and Auto Scaling Groups for automated horizontal scaling using the custom AMI.
- 7 Implemented and configured the Application Load Balancer (ALB) with a Target Group to distribute traffic across multiple EC2 instances in multiple AZs.
- 8 Performed end-to-end application testing and database verification to confirm the complete data flow across the infrastructure.